Assignment 3 — Practice A

Подготовительный вариант A

30 questions • Suggested time: 30 minutes • One correct choice per question.

30 вопросов • Рекомендуемое время: 30 минут • Один правильный ответ.

Read either language. Vertex and input edge IDs start at 0. The answer key and explanations follow after all questions; complete the questions before reading them.

Читайте удобный язык. Вершины и ID рёбер нумеруются с 0. Ответы с объяснениями идут после всех вопросов; сначала решите задания.

These examples practise the same skills as the graded quiz, but are not graded questions. Java fragments use basic arrays, loops and arithmetic.

Примеры проверяют те же навыки, что основной квиз, но не входят в него. Java-фрагменты используют простые массивы, циклы и арифметику.

1. DSU trace

9 venues start as separate components. Apply unions [[1, 2], [3, 4], [2, 3], [1, 4], [5, 5]] in order. What is the final [components, largest_size]?

9 площадок изначально отдельны. Выполните union [[1, 2], [3, 4], [2, 3], [1, 4], [5, 5]] по порядку. Каково итоговое [components, largest_size]?

A. [5, 4]

B. [4, 5]

C. [6, 4]

D. [6, 3]

2. Path compression

The code compresses the existing path to root. What is parent[x] afterwards?

Код сжимает существующий путь к root. Чему после него равно parent[x]?

```
int[] parent = {0, 1, 2, 3, 3, 4, 5};
int root = 3, x = 6;
for (int u = x; u != root; ) {
    int next = parent[u];
    parent[u] = root;
    u = next;
}
```

A. 3

B. 4

C. 5

D. 6

3. Union by size

Which update correctly performs union by size? Sizes are maintained at roots.

Какая правка корректно выполняет union by size? Размеры хранятся у корней.

```
// a and b are DISTINCT roots
size[a] = 17; size[b] = 12;
// Complete union by size here.
```

A. parent[b] = a; size[b] += size[a];

B. parent[a] = b; size[b] += size[a];

C. parent[b] = a; size[a] += size[b];

D. parent[a] = b; parent[b] = a;

4. DSU complexity

DSU uses path compression and union by size (example: n=12000, q=36011). Give general bounds in independent n and q, without assuming a fixed ratio. Which is the standard tight amortized total time and total memory INCLUDING q output states?

DSU использует сжатие путей и union by size (пример: n=12000, q=36011). Дайте общие оценки для независимых n и q, без фиксированного соотношения. Каковы стандартное точное амортизированное время и общая память С УЧЁТОМ q ответов?

A. Time O((n+q) alpha(n)); memory O(n+q). / Время O((n+q) alpha(n)); память O(n+q).

B. Time O(nq); memory O(n+q). / Время O(nq); память O(n+q).

C. Time O((n+q) alpha(n)); memory O(n), including output. / Время O((n+q) alpha(n)); память O(n), включая ответы.

D. Time O(n+q); memory O(1), including output. / Время O(n+q); память O(1), включая ответы.

5. Initialization and reset

A persistent program finishes a DSU request with all 14 vertices connected. A new independent request has n=10. BEFORE processing its operations, what must [components, largest_size] be?

Программа завершила DSU-запрос, объединив все 14 вершин. В новом независимом запросе n=10. Чему ДО обработки его операций должно равняться [components, largest_size]?

A. [10, 11]

B. [1, 14]

C. [10, 1]

D. [14, 0]

6. Kruskal trace

For n=4, undirected edges [u,v,w] are [[0, 1, 12], [1, 2, 15], [0, 2, -12], [2, 3, 11], [0, 1, 15], [3, 3, -60]]. What minimum spanning FOREST cost must Kruskal return?

n=4; неориентированные рёбра [u,v,w]: [[0, 1, 12], [1, 2, 15], [0, 2, -12], [2, 3, 11], [0, 1, 15], [3, 3, -60]]. Какую стоимость минимального остовного ЛЕСА должен вернуть Краскал?

A. 11

B. 38

C. 26

D. -61

7. Ties and edge IDs

n=4, with undirected edges [[0, 1, 12], [1, 2, 12], [0, 2, 12], [2, 2, -30]] in input order (IDs 0..3). Which complete response MUST be accepted, even if a reference solution chooses different edges?

n=4; неориентированные рёбра [[0, 1, 12], [1, 2, 12], [0, 2, 12], [2, 2, -30]] в порядке ввода (ID 0..3). Какой полный ответ ОБЯЗАТЕЛЬНО допустим, даже если эталон выбрал другие рёбра?

A. cost=24, edge_ids=[0,0], components=2

B. cost=24, edge_ids=[1, 2], components=1

C. cost=24, edge_ids=[1, 2], components=2

D. cost=36, edge_ids=[0,1,2], components=2

8. Spanning forest edge count

An undirected graph has n=17 vertices and c=4 connected components, counting isolated vertices. How many edge IDs must a spanning forest contain?

В неориентированном графе n=17 вершин и c=4 связных компонент, включая изолированные вершины. Сколько ID рёбер должен содержать остовный лес?

A. 13

B. 4

C. 16

D. 17

9. Kruskal bound and fair comparison

You compare sort-based Kruskal and Prim at n=[1100, 5500, 11000]. Which plan combines a correct Kruskal time bound with a fair measurement?

Вы сравниваете Краскала с сортировкой и Прима при n=[1100, 5500, 11000]. Какой план сочетает верную сложность Краскала с корректным измерением?

A. Give Kruskal sparse graphs and Prim dense graphs; compare each fastest result. / Краскалу дать разреженные графы, Приму плотные; сравнить лучшие времена.

B. Run each algorithm once on different random inputs and compare only edge IDs. / Запустить один раз на разных случайных входах и сравнить только ID рёбер.

C. Kruskal: O(n+m log(m+1)). Use the same graphs/environment, record n/m and seed, repeat runs and report medians with the same timing boundary. / Краскал: O(n+m log(m+1)). Одинаковые графы/среда, указать n/m и seed, повторить запуски и записать медианы с одинаковой границей измерения.

D. Time compilation for one algorithm but exclude compilation for the other. / Для одного учитывать компиляцию, для другого исключить её.

10. Prim frontier and stale edges

Prim has visited vertices {0,1} in this undirected triangle. Edges [u,v,w] are [[0, 1, -14], [1, 2, -11], [0, 2, -8]]. The heap still contains edge ID 0, whose endpoints are both visited. Which edge ID should be accepted NEXT?

Прим посетил {0,1} в неориентированном треугольнике. Рёбра [u,v,w]: [[0, 1, -14], [1, 2, -11], [0, 2, -8]]. В heap осталось ребро ID 0 с двумя посещёнными концами. Какой ID нужно принять СЛЕДУЮЩИМ?

A. 1

B. 3

C. 0

D. 2

11. Disconnected Prim

For n=16 and undirected edges [[0, 1, 12], [2, 3, -11]], Prim must restart whenever a component is finished. How many components should its final result report?

n=16; неориентированные рёбра [[0, 1, 12], [2, 3, -11]]. Прим перезапускается после окончания каждой компоненты. Сколько компонент должно быть в итоговом ответе?

A. 2

B. 1

C. 14

D. 16

12. Prim versus Kruskal

On n=3 with undirected edges [[0, 1, 13], [1, 2, 13], [0, 2, 13]], Kruskal returns IDs [0, 1] and Prim returns IDs [0, 2]. Both report cost=26, components=1. What does this show?

n=3; неориентированные рёбра [[0, 1, 13], [1, 2, 13], [0, 2, 13]]. Краскал вернул ID [0, 1], Прим ID [0, 2]; оба сообщают cost=26, components=1. Что следует?

A. Both answers are valid minimum spanning trees; ties can produce different IDs. / Оба ответа — допустимые минимальные остовы; равные веса допускают разные ID.

B. Kruskal is wrong because Prim's edges define the only optimum. / Краскал ошибочен: рёбра Прима задают единственный оптимум.

C. Prim is wrong because its IDs must exactly equal Kruskal's IDs. / Прим ошибочен: его ID обязаны совпадать с ID Краскала.

D. Equal costs prove that either answer may contain a cycle or duplicate ID. / Равные стоимости разрешают циклы или повторяющиеся ID в любом ответе.

13. Lazy Prim complexity

Lazy-heap Prim uses adjacency lists, allows parallel edges and scans every component (example: n=5500, m=22010). Give a general worst-case bound for independent n and m, without assuming a fixed ratio. Which includes isolated vertices and heap entries?

Прим с ленивой очередью использует списки смежности, допускает параллельные рёбра и обходит все компоненты (пример: n=5500, m=22010). Дайте общую худшую оценку для независимых n и m, без фиксированного соотношения. Какая учитывает изолированные вершины и элементы heap?

A. Time O(m log(m+1)); memory O(1), even with isolated vertices. / Время O(m log(m+1)); память O(1), даже с изолированными вершинами.

B. Time O(n+m); memory O(n), because the heap can hold at most n edges. / Время O(n+m); память O(n), ведь heap хранит не больше n рёбер.

C. Time O(n+m log(m+1)); memory O(n+m). / Время O(n+m log(m+1)); память O(n+m).

D. Time O(nm); memory O(nm), because an adjacency matrix is required. / Время O(nm); память O(nm), поскольку нужна матрица смежности.

14. SCC partition

Directed graph: n=6, edges=[[4, 5], [5, 4], [5, 0], [0, 1], [1, 0], [1, 2], [4, 4]]. Which partition gives its strongly connected components? Group order is irrelevant.

Ориентированный граф: n=6, рёбра=[[4, 5], [5, 4], [5, 0], [0, 1], [1, 0], [1, 2], [4, 4]]. Какое разбиение задаёт SCC? Порядок групп не важен.

A. [4, 5] | [0, 1] | [2] | [3]

B. [0] | [1] | [2] | [3] | [4] | [5]

C. [0, 1, 4, 5] | [2] | [3]

D. [4, 5] | [0, 1, 2] | [3]

15. Condensation

The directed input edges are [[0, 1], [1, 0], [0, 2], [1, 2], [2, 3], [2, 3], [3, 3]]; SCC labels are [22, 22, 25, 29]. Which condensation edge list is correct? List order is irrelevant.

Ориентированные рёбра: [[0, 1], [1, 0], [0, 2], [1, 2], [2, 3], [2, 3], [3, 3]]; метки SCC: [22, 22, 25, 29]. Какой список рёбер конденсации верен? Порядок списка не важен.

A. [[22, 25], [22, 25], [25, 29]]

B. [[25, 22], [29, 25]]

C. [[22, 25], [25, 29]]

D. [[22, 22], [22, 25], [25, 29], [29, 29]]

16. Kosaraju pass order

Kosaraju's first DFS pass on the ORIGINAL graph appends vertices when they finish; finish=[2, 4, 3, 0, 1]. How should the second pass run?

Первый DFS Косарайю на ИСХОДНОМ графе добавляет вершины при завершении; finish=[2, 4, 3, 0, 1]. Как выполнить второй проход?

A. Traverse the transposed graph, starting unvisited roots in order [1, 0, 3, 4, 2]. / Обходить транспонированный граф, начиная непосещённые корни в порядке [1, 0, 3, 4, 2].

B. Traverse the original graph, starting roots in order [1, 0, 3, 4, 2]. / Обходить исходный граф, начиная корни в порядке [1, 0, 3, 4, 2].

C. Traverse the transposed graph, starting roots in order [2, 4, 3, 0, 1]. / Обходить транспонированный граф, начиная корни в порядке [2, 4, 3, 0, 1].

D. Sort vertices by numerical ID and use the original graph. / Отсортировать вершины по номерам и обходить исходный граф.

17. DFS finishing order

next is the index of the next outgoing neighbour; all earlier neighbours are finished. Which next action preserves genuine DFS finishing order?

next — индекс следующего исходящего соседа; все предыдущие соседи завершены. Какое следующее действие сохраняет настоящий порядок завершения DFS?

```
// First DFS pass; top frame belongs to vertex 12
int next = 2;
int degree = 2;
// If next < degree, that neighbour is WHITE.
```

A. Advance the neighbour index and explore the next WHITE neighbour; do not append this vertex yet. / Продвинуть индекс и обойти следующего WHITE-соседа; текущую вершину пока не добавлять в finish.

B. Clear the whole frame stack and sort vertices by ID. / Очистить весь стек кадров и отсортировать вершины по ID.

C. Append this vertex to finish and pop its completed frame. / Добавить вершину в finish и снять завершённый кадр.

D. Append the vertex now and also append it on every later frame step. / Добавить вершину сейчас и добавлять её повторно на каждом следующем шаге кадра.

18. SCC time and memory

Kosaraju stores adjacency lists and their transpose, and deduplicates condensation with an expected-O(1) hash set (example: n=11000, m=33010). Give the standard general bounds for independent n and m, without assuming a fixed ratio.

Косарайю хранит списки смежности и транспонированный граф, дедуплицирует конденсацию hash set с ожидаемым O(1) (пример: n=11000, m=33010). Дайте стандартные общие оценки для независимых n и m, без фиксированного соотношения.

A. Expected time O(n+m), memory O(n+m). / Ожидаемое время O(n+m), память O(n+m).

B. Time O(m log m), memory $O(n^2)$, because all edges must be sorted. / Время O(m log m), память $O(n^2)$, поскольку нужно сортировать все рёбра.

C. Time O(n+m), memory O(n), even including both adjacency lists and output. / Время O(n+m), память O(n), включая оба списка смежности и результат.

D. Time $O(n^2)$, memory O(1), because SCCs remove all edges. / Время $O(n^2)$, память O(1), поскольку SCC удаляют все рёбра.

19. DFS cycle colours

Why is GRAY the correct condition for detecting a DFS back edge?

Почему условие GRAY правильно обнаруживает обратное ребро DFS?

```
// WHITE=0, GRAY=1 (active), BLACK=2 (finished)
// Explore edge 22 -> 23
if (color[23] == 1) cycle = true;
```

A. A WHITE target already completed its DFS subtree. / WHITE означает, что поддерево цели уже завершено.

B. Only an edge from the source vertex can close a cycle. / Цикл может замкнуть только ребро из исходной вершины.

C. The target is still on the current DFS path; this edge closes a directed cycle. / Цель ещё находится на текущем пути DFS; ребро замыкает направленный цикл.

D. Any previously visited target proves a cycle, including BLACK targets. / Любая уже посещённая цель доказывает цикл, включая BLACK.

20. Topological order

n=4. Directed edges [[2, 0], [3, 0], [0, 1], [2, 1], [3, 1]] mean prerequisite → dependent. Which sequence is a valid topological order of all vertices?

n=4. Ориентированные рёбра [[2, 0], [3, 0], [0, 1], [2, 1], [3, 1]] означают prerequisite → dependent. Какой порядок всех вершин топологически корректен?

A. [3, 2, 0, 1]

B. [2, 3, 1, 0]

C. [0, 2, 3, 1]

D. [3, 0, 2, 1]

21. Cycles in the whole graph

For topo, n=7 and directed edges=[[0, 1], [2, 3], [3, 4], [4, 2]]. A DFS starting at 0 finishes without finding a cycle. What must the complete function return?

Для topo n=7, ориентированные рёбра=[[0, 1], [2, 3], [3, 4], [4, 2]]. DFS из 0 завершился без цикла. Что должна вернуть полная функция?

A. acyclic=true, order=[0,1]

B. acyclic=true, order=[0, 1, 2, 3, 4, 5, 6]

C. acyclic=false, order=[]

D. acyclic=false, order=[0,1]

22. Isolated and empty graphs

For topo, n=10 and edges=[]. Which response satisfies the assignment contract?

Для topo n=10, edges=[]. Какой ответ соответствует контракту задания?

A. acyclic=true, order=[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

B. acyclic=false, order=[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]

C. acyclic=true, order=[9, 8, 7, 6, 5, 4, 3, 2, 1]

D. acyclic=true, order=[9, 8, 7, 6, 5, 4, 3, 2, 1, 0, 10]

23. Iterative DFS complexity

DFS topological ordering must handle a chain of 90000 vertices without recursion-stack failure AND retain O(n+m) traversal time on arbitrary directed graphs. Which strategy guarantees both requirements?

DFS-топологическая сортировка должна обработать цепочку из 90000 вершин без переполнения стека вызовов И сохранять O(n+m) времени на произвольных ориентированных графах. Какой подход гарантирует оба требования?

A. Recursively visit the chain and assume every runtime permits that depth. / Рекурсивно посетить цепочку и считать такую глубину допустимой в любой среде.

B. At every stack step, rescan all neighbours from the beginning. / На каждом шаге стека просматривать всех соседей сначала.

C. Use explicit frames (vertex, next-neighbour index), process each edge once and append a vertex on frame completion. / Использовать явные кадры (вершина, индекс следующего соседа), обрабатывать ребро один раз и добавлять вершину при завершении кадра.

D. Push vertices in numeric order and return that order without DFS. / Положить вершины по номерам и вернуть этот порядок без DFS.

24. DAG distances with negative weights

For the directed DAG n=5, source=0, edges=[[0, 1, 13], [0, 2, 18], [1, 2, -17], [2, 3, 14]], what is distance[3]?

Для ориентированного DAG n=5, source=0, рёбра=[[0, 1, 13], [0, 2, 18], [1, 2, -17], [2, 3, 14]], чему равно distance[3]?

A. 10

B. -3

C. 27

D. 32

25. Unreachable cycle

dag receives n=5, source=0, edges=[[0, 1, 13], [2, 3, -11], [3, 2, 12]]. The cycle cannot be reached from source. Which complete response is required?

dag получает n=5, source=0, рёбра=[[0, 1, 13], [2, 3, -11], [3, 2, 12]]. Цикл недостижим из source. Какой полный ответ требуется?

A. acyclic=false, distance=[0], parent_edge=[-1]

B. acyclic=true, distance=[0, 13, null, null, null], parent_edge=[-1, 0, -1, -1, -1]

C. acyclic=false, distance=[], parent_edge=[]

D. acyclic=true, distance=[], parent_edge=[]

26. Skipping unreachable vertices

Which rule prevents an unreachable vertex from creating a false shortest path?

Какое правило не даст недостижимой вершине создать ложный кратчайший путь?

```
// distance[u] is null: u is unreachable
// edge u -> v has weight -20
Long du = distance[u];
// Select the correct relaxation rule.
```

A. If du == null, skip this edge; otherwise compute and compare du + weight. / Если du == null, пропустить ребро; иначе вычислить и сравнить du + weight.

B. Replace null with zero and relax normally. / Заменить null нулём и выполнить relax.

C. Set distance[v] to weight whenever the weight is negative. / При отрицательном весе присвоить distance[v] значение weight.

D. Replace null with a large sentinel and always add the negative weight. / Заменить null большим значением и всегда прибавлять отрицательный вес.

27. Parent edge witness

For source=0 and directed edges [[0, 1, 12], [0, 2, 16], [1, 2, -13]] (IDs 0,1,2), distance=[0,12,-1]. Which parent_edge array is a valid shortest-path witness?

source=0; ориентированные рёбра [[0, 1, 12], [0, 2, 16], [1, 2, -13]] (ID 0,1,2), distance=[0,12,-1]. Какой parent_edge подтверждает кратчайшие пути?

A. [-1, 0, -1]

B. [-1, 0, 1]

C. [-1, 0, 2]

D. [-1, 2, 2]

28. Source and unreachable route

A DAG has n=6, source=2, only edge 2→3, and isolated vertex 0. Which pair of helper results is required: route(source), route(0)?

В DAG n=6, source=2, единственное ребро 2→3, вершина 0 изолирована. Какие результаты helper требуются: route(source), route(0)?

A. [2], []

B. [2], [0]

C. [], [0]

D. [], []

29. 64-bit arithmetic

A DAG route consists of this many edges of weight w. What does this Java code print? The cast occurs BEFORE multiplication.

Маршрут DAG содержит указанное число рёбер веса w. Что напечатает Java-код? Приведение выполняется ДО умножения.

```
int w = 800000000;
long total = (long) w * 5;
System.out.println(total);
```

A. 800000000

B. -4000000000

C. 4000000000

D. -294967296

30. Route reconstruction

For n=7, source=0, target=3, directed edges=[[0, 1, 4], [0, 2, 9], [1, 2, -6], [2, 3, 3]] and parent_edge=[-1, 0, 2, 3, -1, -1, -1], which vertex route does the helper reconstruct? It follows incoming edge IDs backwards, then reverses the list.

n=7, source=0, target=3, ориентированные рёбра=[[0, 1, 4], [0, 2, 9], [1, 2, -6], [2, 3, 3]], parent_edge=[-1, 0, 2, 3, -1, -1, -1]. Какой маршрут вершин восстановит helper? Он идёт назад по ID входящих рёбер, затем разворачивает список.

A. [0, 1, 2, 3]

B. [0, 1, 3]

C. [0, 2, 3]

D. [3, 2, 1, 0]

Answers and explanations

Ответы и объяснения

Use the explanations to check your reasoning, not only the selected letter. / Сверяйте ход рассуждения, а не только букву.

1. C — DSU trace

Three operations merge distinct components; the redundant union and self-union change nothing. The four linked vertices form the largest component.

Три операции объединяют разные компоненты. Повторное объединение и self-union ничего не меняют; крупнейшая группа содержит четыре вершины.

2. A — Path compression

Every visited non-root vertex is attached directly to the existing root; the component and representative stay the same.

Каждая посещённая нeroot-вершина получает родителем существующий корень; состав компоненты и представитель сохраняются.

3. C — Union by size

Attach the smaller root b under a and update the new root a. Updating a non-root size or creating a parent cycle violates the maintained state.

Меньший корень b присоединяется к a, а размер обновляется у нового корня a. Размер у нeroot или цикл родителей нарушает инвариант.

4. A — DSU complexity

The two heuristics give inverse-Ackermann amortized operations. Parents/sizes need O(n); q returned states need O(q).

Обе эвристики дают амортизированную оценку с обратной функцией Аккермана. Массивы требуют O(n), возвращаемые состояния O(q).

5. C — Initialization and reset

Requests do not share DSU state. A nonempty new request starts with n singleton groups; an empty request starts with zero groups and largest size zero.

Состояние DSU между запросами не сохраняется. Новый непустой запрос начинается с n одиночных групп; пустой — с нулём групп и крупнейшим размером 0.

6. A — Kruskal trace

The cheapest triangle connection uses weights -12 and 12; edge 2-3 adds 11. Self-loops cannot belong to a forest, and any isolated vertex costs zero.

Самая дешёвая связь внутри треугольника использует веса -12 и 12; ребро 2-3 добавляет 11. Self-loop не входит в лес, изолированная вершина стоимости не добавляет.

7. C — Ties and edge IDs

Any two distinct triangle edges form a minimum tree; vertex 3 is isolated. An optimal forest need not match reference edge IDs. Duplicate IDs, a cycle or an incorrect component count are invalid.

Любые два разных ребра треугольника образуют минимальное дерево; вершина 3 изолирована. ID могут отличаться от эталона. Повтор ID, цикл и неверное число компонент недопустимы.

8. A — Spanning forest edge count

A tree on k vertices has k-1 edges. Summing across c components gives n-c, including zero edges for singleton components.

В дереве из k вершин k-1 рёбер. Сумма по c компонентам равна n-c; одиночная вершина даёт ноль рёбер.

9. C — Kruskal bound and fair comparison

Sorting m edges contributes O(m log(m+1)), while initialization contributes O(n). Change the algorithm, not the graph/environment or timing boundary. Repeat runs and report medians; optimal costs are checked separately because tied IDs may differ.

Сортировка m рёбер даёт O(m log(m+1)), инициализация O(n). Меняется алгоритм, а не граф/среда/граница измерения. Нужны повторы и медианы; оптимальные стоимости сверяются отдельно, поскольку ID могут отличаться.

10. A — Prim frontier and stale edges

Discard the stale edge 0. Among edges crossing from {0,1} to unvisited vertex 2, edge 1 has the smaller weight.

Устаревшее ребро 0 пропускается. Из рёбер между {0,1} и непосещённой вершиной 2 меньший вес у ребра 1.

11. C — Disconnected Prim

Two disjoint edges each merge two vertices; the remaining vertices are isolated and each counts as one component.

Два непересекающихся ребра каждое объединяют пару вершин; остальные вершины изолированы и считаются отдельными компонентами.

12. A — Prim versus Kruskal

Both listed pairs are distinct, acyclic, spanning and optimal. Equal cost alone would not validate an arbitrary malformed witness.

Обе пары различны, ацикличны, охватывают граф и оптимальны. Само равенство стоимости не оправдывает некорректное свидетельство.

13. C — Lazy Prim complexity

A lazy heap can hold O(m) edge candidates. Heap operations add log(m+1), while initializing/scanning vertices adds O(n). Adjacency lists require O(n+m).

В ленивом heap возможно O(m) кандидатов. Операции очереди добавляют log(m+1), инициализация и проход по вершинам O(n); списки смежности занимают O(n+m).

14. A — SCC partition

Each two-way pair is an SCC. One-way links between pairs do not make them mutually reachable; the final sink and isolated vertex are singleton SCCs.

Каждая двусторонняя пара — SCC. Односторонние связи не создают взаимной достижимости; конечная вершина и изолированная вершина образуют одиночные SCC.

15. C — Condensation

Discard edges inside each SCC and deduplicate directed component pairs. The repeated links become one edge per pair; directions are preserved.

Внутренние рёбра SCC удаляются, направленные пары компонент дедуплицируются. Повторы дают одно ребро на пару; направления сохраняются.

16. A — Kosaraju pass order

With the first pass on G, the second pass uses G transpose and decreasing finishing time. Each new DFS root identifies one SCC.

После первого прохода на G второй использует транспонированный граф и убывающий порядок завершения. Каждый новый корень DFS задаёт одну SCC.

17. C — DFS finishing order

A vertex finishes only when next equals its degree. If an unprocessed neighbour remains, advance the saved index and explore that neighbour before finishing the current frame.

Вершина завершается только при next=degree. Пока есть необработанный сосед, нужно продвинуть сохранённый индекс и обойти соседа, не завершать текущий кадр.

18. A — SCC time and memory

Two DFS passes and edge scanning are linear. The graph, transpose, traversal state and up to O(m) condensation pairs fit O(n+m) memory; hashing makes deduplication expected linear time.

Два DFS и проход по рёбрам линейны. Граф, транспонированный граф, состояние обхода и до O(m) пар занимают O(n+m); hash set даёт ожидаемую линейную дедупликацию.

19. C — DFS cycle colours

GRAY identifies an ancestor on the active call/frame stack. A BLACK target may be a harmless forward/cross edge and is not by itself proof of a cycle.

GRAY обозначает предка на активном стеке. BLACK может быть целью безопасного прямого/поперечного ребра и само по себе цикл не доказывает.

20. A — Topological order

Both prerequisites must precede the shared dependent, and that vertex must precede its successor. The two independent prerequisites may appear in either order.

Обе предпосылки должны идти до общей зависимой вершины, а она до следующей. Две независимые предпосылки могут располагаться в любом порядке.

21. C — Cycles in the whole graph

Topo must inspect every component. A directed cycle or self-loop elsewhere makes a topological ordering impossible; the contract requires an empty order when cyclic.

Topo проверяет все компоненты. Цикл или self-loop в другой области исключает топологический порядок; при цикле контракт требует пустой order.

22. A — Isolated and empty graphs

An edgeless graph is acyclic; every vertex must appear exactly once. Any permutation works, including the empty list when n=0.

Граф без рёбер ацикличен; каждая вершина появляется ровно один раз. Подходит любая перестановка, при n=0 пустой список.

23. C — Iterative DFS complexity

Frames replace recursive state and remember progress. They take O(n) traversal memory; together with adjacency lists the implementation uses O(n+m) time/memory.

Кадры заменяют рекурсивное состояние и запоминают прогресс. Сам стек занимает O(n); со списками смежности время и память O(n+m).

24. A — DAG distances with negative weights

The route through both intermediate vertices costs 13+(-17)+14=10, less than the direct branch followed by the last edge. Negative weights are valid in a DAG.

Маршрут через две промежуточные вершины стоит 13+(-17)+14=10 и дешевле прямой ветки с последним ребром. Отрицательные веса допустимы в DAG.

25. C — Unreachable cycle

The assignment requires the whole graph to be acyclic, not only the reachable subgraph. On any cycle, all distance/parent arrays are empty.

Задание требует ацикличности всего графа, не только достижимой области. При любом цикле distance и parent_edge пусты.

26. A — Skipping unreachable vertices

No route reaches u, so no route through this edge is known. Relaxing a null/infinite distance can create fake reachability or overflow.

Путь до u отсутствует, поэтому через это ребро путь не возникает. Relax от null/бесконечности может дать ложную достижимость или переполнение.

27. C — Parent edge witness

Edge 0 ends at vertex 1 and gives its distance. Edge 2 ends at vertex 2 and gives the optimal path through vertex 1; direct edge 1 has a larger cost.

Ребро 0 заканчивается в вершине 1 и даёт её расстояние. Ребро 2 заканчивается в вершине 2 и задаёт оптимальный путь через 1; прямое ребро 1 дороже.

28. A — Source and unreachable route

The source has distance zero and parent_edge -1, so its route is [source]. The unreachable vertex has null distance and parent_edge -1, so its route is [].

Источник имеет расстояние 0 и parent_edge=-1, его маршрут [source]. Недостижимая вершина имеет distance=null и parent_edge=-1, её маршрут [].

29. C — 64-bit arithmetic

The left operand is converted to long before multiplication, so the sum is 4000000000 without 32-bit overflow. Casting an already-overflowed int result afterwards would be too late.

Левый операнд преобразуется в long до умножения, поэтому сумма 4000000000 не переполняет 32 бита. Приведение уже переполненного int-результата было бы поздним.

30. A — Route reconstruction

Follow the tail of the incoming edge from target back to source, then reverse. The work is proportional to the route length, not a new shortest-path search; a tied alternative is valid only with consistent parent edges.

От target нужно идти к началу входящего ребра до source, затем развернуть список. Работа пропорциональна длине пути, это не новый поиск кратчайшего пути; альтернативный оптимум требует согласованных parent_edge.